



Final project

Report

SMART NURSERY GUARDIAN

An AI-Assisted Embedded Monitoring System for Infant Care

1. Introduction

At 3 a.m., a baby cries. Is she hungry? Tired? The parent stands in darkness guessing. Crying is the baby's only language, and most parents cannot read it fluently.

Smart Nursery Guardian listens to the cry, uses machine learning to identify the cause (hungry, tired, or discomfort), and reacts instantly: rocking the crib, displaying a clear message, or raising a safety alert. Nearby sensors monitor motion (is the baby awake?), light level, air quality, and temperature, controlling lights and cooling automatically.



The system aims to respond as fast and attentively as a caregiver watching the crib.

The result is a system that responds like an attentive caregiver, but never sleeps and never misses a signal.

1.1 Why This Project Matters

- Addresses a real problem: sleep loss from misinterpreting infant cries, solved by turning guesswork into data-backed guidance.
- Realistic technical scope: signal processing, machine learning, real-time firmware, GUI design, and IoT communication.

2. How the System Works

This section describes step by step what happens when the system detects a cry or motion, so you can picture the entire sequence.

2.1 Listening for Cries (Laptop Microphone)

The laptop's microphone runs a continuous audio pipeline in the background:

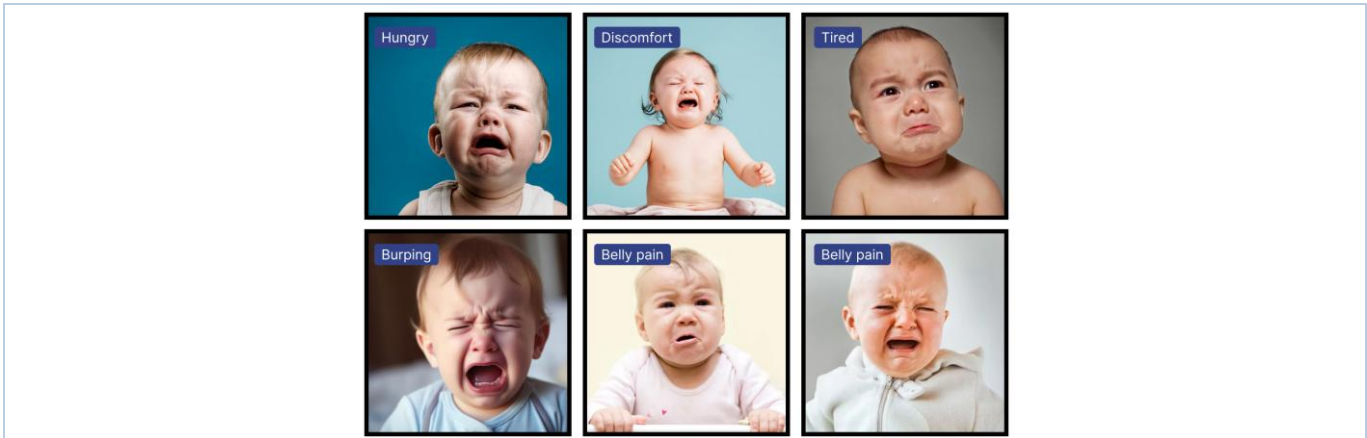
- Capture rolling audio stream.
- Remove background noise (hum, fans) using spectral filtering.
- Run voice activity detection to confirm a real cry is present.
- Trigger servo rocking and alert the microcontroller immediately.
- Extract cry features and pass them to the trained ML model to classify (hungry, tired, or discomfort).

2.2 Instant Rocking (Servo Motor)

The moment a cry is confirmed, the servo rocks the crib continuously back and forth (0, 90, -90 degrees in a cycle). This is the same first instinct a caregiver has: soothe before analyzing. The rocking stops when the cry ends or the classification response takes over.

2.3 Classification Results

- Hungry cry: display message and play calming video clip.
- Tired cry: sound buzzer immediately and display warning (this pattern is urgent).
- Discomfort cry: display message and suggest checking diaper, clothing, position.



Cry patterns differ by cause - the model learns to tell them apart from audio features alone.

NOTE

All three responses assume rocking is already running from Section 2.2.

2.4 Motion Detection (PIR Sensor)

A motion sensor near the crib counts movements. If 4 or more motions occur within an 8-second window, the system concludes the baby is awake. This threshold prevents false alarms from hands passing nearby or blankets shifting.

2.5 Room Lighting (Light Sensor + LED)

The LED (room lamp) turns on only when the room is dark AND one of these is true:

- Baby is awake (4+ motions in 8 seconds detected), OR
- A cry is currently being detected.

The laptop sends cry-detected and cry-ended signals to the microcontroller so the LED logic can combine them with local motion and light readings.

2.6 Safety Alerts (Gas Sensor)

If gas or smoke is detected, the buzzer sounds continuously, the GUI shows a red full-screen alert, and a Telegram message is sent to the caregiver's phone. This is the highest-priority alert.



Gas or smoke detection triggers the system's highest-priority alert path.

2.7 Temperature Control (Thermistor)

Temperature Range	Status Light Color	Cooling Fan
Below 25°C	Blue	Off
25°C to 30°C	Green	Half speed
Above 30°C	Red	Full speed

The color and fan speed change together from the same temperature reading, so what the caregiver sees always matches what the fan is doing.

3. Software Architecture

Two programs run on two devices and communicate over serial:

Part	Runs On	Responsibility
Python App (Tkinter GUI)	Laptop	Microphone input, noise filter, cry detection, ML classification, display messages, play videos, send Telegram alerts
Firmware	Microcontroller	Read PIR, light, temperature, gas sensors; control servo, buzzer, LED, fan; relay sensor events to laptop

Two critical data paths work differently:

- Microphone: Laptop reads its own mic directly into Python. No microcontroller involvement.
- All other sensors (motion, light, temperature, gas): Microcontroller reads them and sends values to Python over serial.

HINT

This asymmetry matters: audio never crosses serial, so cry detection has zero latency. Other sensors must travel through the microcontroller, so optimize serial messages to avoid flooding.

The GUI is built with Tkinter, Python's standard library, chosen because it needs no extra installation and is sufficient for status displays, warnings, and video playback.

3.1 Why Split Work This Way

- AI, audio processing, and GUI need Python's libraries and computing power. Microcontrollers cannot do this.
- Real-time sensor and motor control need the microcontroller's low-level timing precision.
- This pattern (edge device for sensing/actuation + smart device for decisions) is a standard real-world IoT design you will see professionally.

4. Machine Learning Pipeline



Turning a raw cry waveform into features a model can learn from.

4.1 The Approach

Collect labeled cry recordings. Extract numerical features from each (MFCC, pitch, duration, intensity). Train a model on these numbers and labels. Test on unheard-before recordings. Deploy the saved model into the live system.

HINT

Treat training and testing like you treat hygiene: keep them separate. Never test on data you trained on, ever. Split 80 percent training, 20 percent testing before any augmentation or training begins.

4.2 The Data Problem: Hungry Dominates

Class	Recordings	Percent
Hungry	382	84 percent
Discomfort	27	6 percent
Tired	24	5 percent

If you train a model on this raw data, it learns to guess hungry every single time and achieves 84 percent accuracy while being useless at detecting tired or discomfort. This is a real problem documented in published research on this exact dataset.

Solution: Data augmentation. Create synthetic copies of tired and discomfort recordings by slightly transforming the originals.

- Pitch shift: move pitch up or down by 2 semitones or less.
- Time stretch: play at 0.9x to 1.1x speed without changing pitch.

- Add noise: mix in light background noise so the model does not memorize silence.

Apply these only to training data, never to test data. Published research using this method on this exact dataset raises minority-class accuracy from near zero to over 90 percent.

HINT

Always check a confusion matrix (table of predictions vs. actual labels) per class, not just overall accuracy. A model that only ever predicts hungry will still score high on blind overall accuracy.

4.3 Cleaning the Audio

Raw laptop microphone audio is noisy. Two stages clean it:

Stage 1: Spectral noise reduction. Record the room's quiet background. Convert audio to a spectrogram (frequency over time). Suppress frequencies matching the background. The noisereduce Python library does this.

Stage 2: Voice activity detection (VAD). Analyze short 20-30ms audio frames. Decide if each frame contains real sound or silence. The webrtcvad library is lightweight and needs no GPU.

HINT

Only start feature extraction and ML after VAD confirms sound. This avoids wasting computation on silence and prevents false detections from random clicks.

4.4 Feature Extraction

The model cannot understand raw audio. Extract numbers that describe the sound:

Feature	What It Captures
MFCC	Tone color and perceived pitch, how the human ear hears it.
Duration	How long the cry lasts.
Intensity (RMS)	Average loudness.
Fundamental Frequency (F0)	Base pitch. Infant cries range 400-500 Hz.
Zero-Crossing Rate (ZCR)	How harsh the sound is.

For classical models, compute mean and standard deviation of each MFCC coefficient across the entire cry, plus duration, intensity, pitch. This gives one fixed-size row per recording, which is what classical models need.

HINT

Start with MFCC alone. It performs close to best-published results by itself. Add other features and measure if accuracy actually improves. Do not add complexity without proof of benefit.

4.5 Where to Get Data

Donate-A-Cry Corpus. 457 labeled recordings, publicly available, built for exactly this task. Use this instead of recording your own.

GitHub: github.com/gveres/donateacry-corpus

Kaggle: kaggle.com/datasets/warcoder/infant-cry-audio-corpus

For faster start, pre-extracted features exist:

Kaggle Features: kaggle.com/datasets/bhoomikavalani/donateacrycorpusfeaturesdataset

HINT

Write your own feature extraction code at least once, even if you use pre-extracted features to skip ahead. That is where the learning happens.

4.6 Choosing a Model

Option A (Recommended): Classical machine learning. Random Forest or SVM. Trains in seconds on a laptop. No GPU needed. Easy to debug. Published research on this exact dataset: 96 percent accuracy with MFCC-only Random Forest.

Option B (Extension): Deep CNN on spectrograms. Can learn patterns hand-picked features miss. Needs more data and time. Harder to debug when wrong.

HINT

Avoid overfitting: your training accuracy is 99 percent, but test accuracy is 70 percent? You memorized the training data. Use cross-validation, early stopping, or data augmentation to fix this. Avoid underfitting: both training and test accuracy are 60 percent? Model is too simple or learning poorly. Increase model complexity or train longer.

4.7 Training Checklist

- Split 80/20 before any augmentation.
- Augment only training data, never test data.
- Check confusion matrix per class.
- Evaluate on unseen test data only.
- Save the model to a file so the live system loads it once and reuses it, never retraining.

4.8 Where Training Runs

On the laptop in Python. The microcontroller has no memory or power for ML. Once trained, the saved model loads once when the Python app starts. Every new cry runs through the already-trained model instantly.

5. System Flow (Complete Sequence)

5.1 Startup

1. Microcontroller powers on. Initialize PIR, light sensor, temperature sensor, gas sensor.
2. Python app starts on laptop. Load trained ML model. Open serial connection to microcontroller.
3. Tkinter GUI opens in calm state. Begin listening to microphone.

5.2 Main Loops (Continuous)

Laptop and microcontroller run independent loops. They communicate only when needed, not on every reading, to avoid flooding the serial connection.

Laptop Loop:

1. Capture microphone audio in chunks.
2. Apply noise reduction.
3. Run voice activity detection.
4. If no cry, repeat.
5. If cry detected, send cry-detected signal to microcontroller (starts servo, enables LED cry-state) and keep signal active.
6. Extract features and run through trained ML model.
7. Display result: hungry (with video), tired (with buzzer), or discomfort (with message).
8. When cry ends, send cry-ended signal to microcontroller (stop servo, clear LED cry-state).

Microcontroller Loop:

1. Read PIR, light, temperature, gas sensors.
2. Count PIR motions in 8-second window. If 4+, report baby awake to laptop.
3. Decide LED: room dark AND (baby awake OR cry detected from laptop). Turn on if both conditions true.
4. Decide fan and status light from temperature alone. No laptop input needed.
5. If gas threshold exceeded, sound buzzer and report to laptop.
6. When laptop sends commands (cry-detected, servo start/stop, buzzer), execute immediately.

NOTE

Fan and status light are completely local (temperature alone). LED is the exception: it runs on the microcontroller but depends on cry-detected from the laptop.

5.3 Serial Messages

Direction	Message Purpose
Microcontroller to Laptop	Baby awake (4+ motions in 8 sec)
Microcontroller to Laptop	Gas or smoke alert
Microcontroller to Laptop	Temperature reading (periodic)

Direction	Message Purpose
Laptop to Microcontroller	Cry detected / Cry ended
Laptop to Microcontroller	Servo start / Servo stop
Laptop to Microcontroller	Buzzer on (tired cry alert)

6. Telegram Notifications

When gas is detected, the system sends a Telegram message to a pre-configured chat so the caregiver gets a phone alert even from another room. There are two ways to implement this:



A lightweight bot relays alerts straight to the caregiver's phone.

Option A: Telegram Bot (BotFather)

Create a bot through Telegram's BotFather, obtain a token and chat ID, then use the python-telegram-bot library to send messages. No custom networking code needed.

Setup: Message @BotFather on Telegram, send /newbot, and get an API token. Get your chat ID by messaging the bot then checking <https://api.telegram.org/bot<TOKEN>/getUpdates>. Then pip install python-telegram-bot and send alerts with a few lines of code.

Cost: Free. No usage fees, no server needed, Telegram hosts everything.

Option B: Telegram Web Automation Scenario

Instead of a bot, automate the Telegram Web interface (web.telegram.org) using a browser-automation tool like Selenium or Playwright to open the chat and send the message when gas is detected.

Trade-offs: No bot account needed, but slower, heavier (needs a running browser), and more fragile if Telegram changes its web layout.

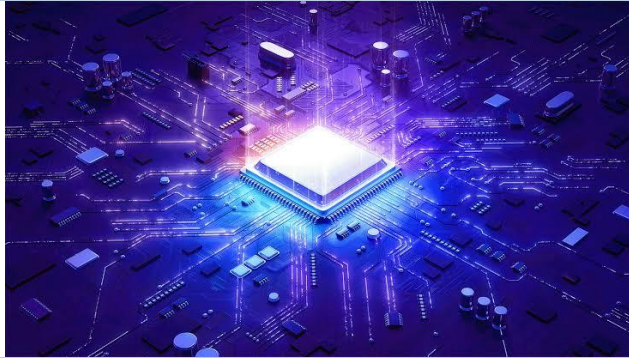
Cost: Also free, but more setup effort and less reliable than the bot.

RECOMMENDATION

Option A is simpler and more reliable.

7. Hardware Part

This section covers the physical build of the Smart Nursery Guardian: the components used on the breadboard prototype, and the PCB design task in Altium that turns the prototype into a manufacturable board.



The microcontroller sits at the center of every sensor and actuator connection.

7.1 Components

- Stm BlackPill (or any suitable controller available to you).

Link of STM: makerselectronics.com/product/stm32f401rct6-stm32-development-board

- Temperature sensor (NTC thermistor) (or any other sensor that does the same job).
- PIR sensor (or any sensor that does the same job).
- Photoresistor sensor (LDR).
- Buzzer.
- RGB LED.
- Normal LED for lighting.
- Motor Driver L293D IC (or any suitable substitute).
- DC Motor + Fan Blades.
- Servo motor (microservo).
- Gas Sensor.
- Breadboard.

If you want to add any other component, feel free to do so.

If you have any trouble soldering the STM, you can visit us at our workshop, but make sure to contact a Hardware member before coming.

HINTS

Make sure you understand the role of each sensor and actuator before starting the hardware design. Think about which components need GPIO, ADC, PWM, or other suitable interfaces. Pay attention to power requirements, especially for the motor, servo, LEDs, and sensors. Keep the sensor signals and motor/power connections organized. Make sure the STM32 pins you select are suitable for the required function.

7.2 Altium Task

A PCB must be designed for the project using the components above, with a slight change: the design should use a STM32 IC like the one used in the PCB2 session and Microcontroller HW session. An existing old design may be reused and built upon, incorporating it into the new design.



Altium Designer is used to turn the breadboard prototype into a manufacturable 4-layer PCB.

The board should be 4 layers with proper planes and tracks, using the design rules and constraints used in JLCPCB.

The design should contain 4 files:

- schdoc
- pcbdoc
- schlib
- pcblib

NOTES

Don't forget to use the assembly notes layer. Use any supplier, but don't forget the 4 parameters: Manufacturer, Manufacturer Part Number, Supplier, and Supplier Part Number.

HINTS

Make sure the schematic and PCB are properly synchronized. Keep the component placement clean and practical. Use proper power and ground planes for the 4-layer board. Check clearances, track widths, vias, and other design rules before finalizing. Make sure all components have the required supplier information. Check the PCB in 3D view and run the design rule check before submission.

8. Summary

This system combines cry classification, sensor monitoring, and coordinated hardware control. The laptop listens to the baby and makes intelligent decisions. The microcontroller handles real-time hardware tasks and relays sensor data. Together they react to the baby like an attentive caregiver, but with instant recognition and zero fatigue.

— End of Report —